SECURITY

SAST & DAST Visual Guide

Static + Dynamic testing —
find bugs in code and in running apps.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

SAST: Reading the Code

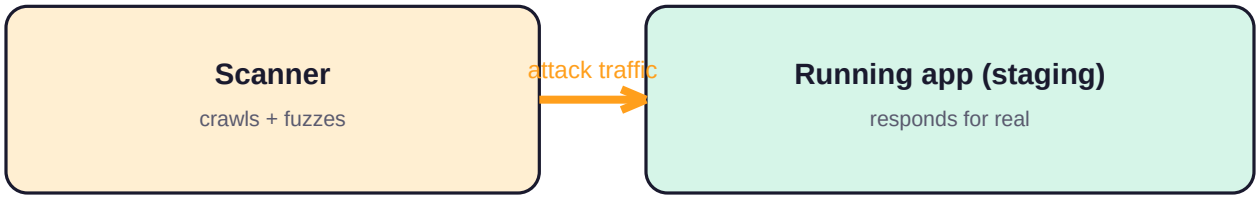
Static analysis — no execution, just reasoning about source.

SAST finds: injection flaws, hardcoded secrets, insecure crypto, taint-flow bugs. Fix cost rises 10x+ after merge — catch it in the IDE.

STEP 2/6

DAST: Attacking the Running App

Dynamic analysis — real HTTP against a live deployment.

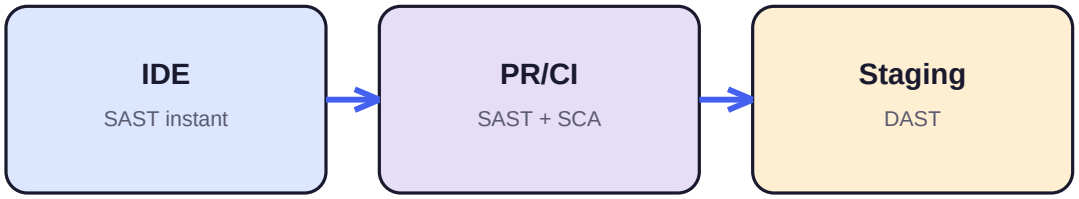


DAST finds: auth bypasses, session issues, server misconfig, XSS/SQLi that only manifest at runtime. Needs a faithful staging environment.

STEP 3/6

Shift-Left Timeline

When each technique pays off.



Cost of a fix grows left→right. SAST everywhere, DAST on every staging deploy, pentest on major releases.

STEP 4/6

Don't Forget SCA

Software composition analysis — SAST for your dependencies.

- SCA scans lockfiles against CVE databases on every build.
- Prioritize reachable + exploitable vulns over raw CVE counts.
- Auto-PR safe upgrades; track the rest with SLAs by severity.
- SBOM out, CVE feed in — SCA closes the loop on supply chain.

STEP 5/6

Triage Without Drowning

Scanners are noisy — tune the signal.

- Break builds only on high-confidence, high-severity findings at first.
- Baseline existing issues; block only NEW findings to avoid big-bang cleanup.
- Dedupe across tools; route to the owning team with fix guidance attached.
- Track false-positive rates per rule — disable or tune chronic noisemakers.

STEP 6/6

The Combined Pipeline

One flow, all three scanners.

PR → SAST + SCA gate (block on new highs)

developer feedback in minutes

Staging deploy → DAST scan (block on criticals)

runtime truth before prod

Release → pentest sample + continuous rescan

humans where automation stops

STEP ✓/✓

Key Takeaways

SAST & DAST — Remember

- SAST reads code (IDE/CI) — cheapest fixes; DAST attacks running apps (staging) — runtime truth.
- Shift left: the earlier the find, the cheaper the fix.
- Add SCA for dependencies — vulnerable libraries are the top finding source.
- Baseline and block only new findings; tune noisy rules instead of ignoring them.
- Pipeline: SAST+SCA on PRs, DAST on staging, pentest on releases.
- No scanner replaces threat modeling and manual review for logic flaws.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev